

Run a MongoDB Vector Search Query

[Copy page](#) ▾

Rate this page



On this page

Overview

[MongoDB Vector Search Index](#)[Vector Search Aggregation Stage](#)[MongoDB Search Query Examples](#)[Basic Vector Search Query](#)[Vector Search Score](#)[Vector Search Options](#)[Additional Information](#)[API Documentation](#)

Overview

In this guide, you can learn how to perform searches on your documents by using the MongoDB Vector Search feature. The C++ driver allows you to perform MongoDB Vector Search queries by using the aggregation framework. To learn more about performing aggregations, see the [Aggregation](#) guide.

NOTE

Deployment Compatibility

You can use the MongoDB Vector Search feature only on MongoDB Atlas clusters. This feature is not available for self-managed deployments.

To learn more about MongoDB Vector Search, see the [MongoDB Vector Search Overview](#) in the Atlas documentation.

NOTE

MongoDB Search

To perform advanced full-text search on your documents, you can use the MongoDB Search feature. To learn about this feature, see the [MongoDB Search Overview](#).

MongoDB Vector Search Index

Before you can perform MongoDB Vector Search queries, you must create a MongoDB Vector Search index on your collection. To learn more about creating this index, see the [MongoDB Vector Search Indexes](#) guide.

Vector Search Aggregation Stage

To create a `$vectorSearch` stage in your aggregation pipeline, perform the following actions:

1. Create a vector to store the pipeline stages.
2. Specify the `$vectorSearch` operator and provide details about the vector search query.

You must define the following fields in your `$vectorSearch` stage:

Parameter	Data Type	Description
<code>index</code>	string	Name of the vector search index
<code>path</code>	string	Field that contains vector embeddings
<code>queryVector</code>	BSN::Binary vector subtype, BSN::Vector, or array of numbers	Vector representation of your query
<code>limit</code>	number	Number of results to return

Then, include the stage in an aggregation pipeline and pass the pipeline to the `aggregate` method.

MongoDB Search Query Examples

In this section, you can learn how to perform MongoDB Vector Search queries. The examples in this section use sample data from the `sample_mflix.embedded_movies` collection. To learn how to load this sample data, see the [Load Data into Atlas](#) tutorial in the Atlas documentation.

Basic Vector Search Query

The following code performs a MongoDB Vector Search query on the `plot_embedding` vector field by using a query vector that is a vector embedding of the phrase "time travel":

```

#include <bsoncxx/builder/basic/document.hpp>
#include <bsoncxx/builder/list.hpp>
#include <bsoncxx/json.hpp>
#include <mongocxx/client.hpp>
#include <mongocxx/instance.hpp>
#include <mongocxx/uri.hpp>

using bsoncxx::builder::basic::kvp;
using bsoncxx::builder::basic::make_document;

int main() {
    try {
        mongocxx::instance inst{};

        // Replace the placeholder with your Atlas connection string
        const auto uri = mongocxx::uri{"<connection-string>"};

        // Connect to your Atlas cluster
        auto client = mongocxx::client{uri};
        auto db = client["sample_mflix"];
        auto collection = db["embedded_movies"];

        // Define vector embeddings to search
        auto vectors = bsoncxx::builder::list{
            -0.034731735, 0.008558298, -0.0153717, -0.029912498, 0.011549547, 0.0102616
        };

        // Define the pipeline with vectorSearch query options
        mongocxx::pipeline stages;

        stages
            .append_stage(make_document(
                kvp("$vectorSearch",
                    make_document(kvp("index", "vector_index"),
                                   kvp("path", "plot_embedding_voyage_3_large"),
                                   kvp("queryVector", vectors),
                                   kvp("numCandidates", 150), kvp("limit", 10))))
            .project(make_document(
                kvp("_id", 0), kvp("plot", 1), kvp("title", 1)));

        auto cursor = collection.aggregate(stages);

        for (auto&& doc : cursor) {
            std::cout << bsoncxx::to_json(doc) << std::endl;
        }
    } catch (const std::exception& e) {
        std::cout << "Exception: " << e.what() << std::endl;
    }
    return 0;
}

```

VIEW OUTPUT

This query uses the `$vectorSearch` stage to:

- Perform an Approximate Nearest Neighbor (ANN) vector search
- Search for the specified term in the `plot_embedding_voyage_3_large` field
- Set the number of nearest neighbors used in the search to 150 by using the `numCandidates` option
- Return a maximum of 10 documents from the query using the `limit` option

It uses the `$project` stage to:

- Only include the movie `plot` and `title` fields in the results

Vector Search Score

The following code performs the same query as in the preceding example, but uses the `$project` stage to output only the `title` field and `vectorSearchScore` meta field, which describes how well the document matches the query vector:

```

// Define the pipeline with vectorSearch and project stages
mongocxx::pipeline score_pipeline;

```

```
score_pipeline
  .append_stage(make_document(
    kvp("$vectorSearch",
      make_document(kvp("queryVector", vectors),
        kvp("index", "vector_index"),
        kvp("path", "plot_embedding_voyage_3_large"),
        kvp("numCandidates", 150),
        kvp("limit", 10))))))
  .project(make_document(
    kvp("_id", 0),
    kvp("title", 1),
    kvp("score", make_document(kvp("$meta", "vectorSearchScore"))));

auto cursor = collection.aggregate(score_pipeline);

for (auto&& doc : cursor) {
  std::cout << bsoncxx::to_json(doc) << std::endl;
}
```

VIEW OUTPUT

Vector Search Options

You can use a `$vectorSearch` stage to perform many types of Atlas Vector Search queries. Depending on your desired query, you can specify the following options in the stage definition:

Optional Parameter	Data Type	Description	Default Value
<code>exact</code>	boolean	Specifies whether to run an Exact Nearest Neighbor (<code>true</code>) or Approximate Nearest Neighbor (<code>false</code>) search	<code>false</code>
<code>filter</code>	document	Specifies a pre-filter for documents to search on	No filtering on
<code>numCandidates</code>	number	Specifies the number of nearest neighbors to use during the search	No limit

To learn more about these options, see the [Fields](#) section of the `$vectorSearch` operator reference in the Atlas documentation.

Additional Information

To learn more about the concepts mentioned in this guide, see the following Server manual entries:

- [Run Vector Search Queries](#)
- [Aggregation Pipeline](#)
- [Aggregation Stages](#)

To learn more about the behavior of the `aggregate` method, see the [Aggregation](#) guide.

API Documentation

To learn more about the methods and types mentioned in this guide, see the following API documentation:

- [aggregate\(\)](#)

